Checklist : Gestion et optimisation de l'occupation des lits hospitaliers

> Comment l'utiliser : place ce fichier dans `docs/` de ton projet et coche les cases (`[ ]` devient `[x]`) dans VS Code, PyCharm, Obsidian ou GitHub. Ordre à respecter : base de données, puis modèle (avec tests), puis contrôleur, puis vue. Teste chaque couche avant de passer à la suivante.

Suivi global

- [ ] Phase 0 : Préparation
- [ ] Phase 1 : Conception
- [ ] Phase 2 : Base de données MySQL
- [ ] Phase 3 : Modèle, entités simples
- [ ] Phase 4 : Modèle, séjours et mouvements
- [ ] Phase 5 : Modèle, indicateurs
- [ ] Phase 6 : Modèle, alertes
- [ ] Phase 7 : Modèle, simulation
- [ ] Phase 8 : Jeu de données fictives
- [ ] Phase 9 : Tests automatisés (bilan)
- [ ] Phase 10 : Contrôleur
- [ ] Phase 11 : Vue PyQt
- [ ] Phase 12 : Intégration et durcissement
- [ ] Phase 13 : Extensions facultatives
- [ ] Phase 14 : Documentation et rendu

Phase 0 : Préparation (½ journée)

0.1 Environnement

- [x] Installer Python 3
- [x] Installer MySQL Server et vérifier que je peux m'y connecter
- [x] Créer un environnement virtuel (`python -m venv venv`)
- [x] `pip install PyQt5 mysql-connector-python matplotlib pytest`
- [x] Créer `requirements.txt` (`pip freeze > requirements.txt`)

0.2 Dépôt et arborescence

- [x] Créer le dépôt Git (et un `.gitignore` : `venv/`, `__pycache__/`, mot de passe MySQL)
- [x] Créer l'arborescence :

```
projet_lits/
├── main.py
├── config.py              (connexion MySQL, seuils par défaut)
├── sql/schema.sql
├── model/                 (database.py, exceptions.py, service_model.py,
lit_model.py, ...)
├── controller/
├── view/
├── data/seed.py           (jeu de données fictives + reset)
├── tests/
└── docs/                  (conception, déclaration IA, manuel)
```

- [ ] Ajouter un fichier `__init__.py` dans `model/`, `controller/`, `view/`, `data/`, `tests/`
- [ ] Faire un premier commit

0.3 Déclaration IA (à tenir à jour tout le long)

- [ ] Créer `docs/declaration_ia.md` avec les colonnes : tâche concernée / suggestion retenue / vérification faite
- [ ] Prendre l'habitude de le remplir après chaque usage significatif de l'IA (conception BDD, génération de code, résolution d'erreurs, création de tests)

0.4 Conventions à fixer par écrit (dans `docs/conventions.md`)

- [ ] Format de date unique : `AAAA-MM-JJ HH:MM:SS`
- [ ] Journée-lit = durée en heures ÷ 24
- [ ] États des lits sans accents dans le code : `DISPONIBLE`, `OCCUPE`, `INDISPONIBLE`
- [ ] Statuts des séjours : `EN_COURS`, `TERMINE`
- [ ] Types de mouvement : `ADMISSION`, `TRANSFERT`, `SORTIE`
- [ ] Nom normalisé d'un service : `strip()` + minuscules + suppression des accents
- [ ] Règle de dépendance MVC : la vue appelle le contrôleur, le contrôleur appelle le modèle, le modèle n'importe jamais PyQt, la vue n'importe jamais le modèle

---

Phase 1 : Conception (1 à 2 jours)

> Ne code rien avant d'avoir fini cette phase.

1.1 Règles métier

- [ ] Créer `docs/regles_metier.md`, une règle par ligne et numérotée (R1, R2, ...)
- [ ] Règles Services : nom unique après normalisation (R)
- [ ] Règles Services : seuil entre 1 et 100 (R)
- [ ] Règles Services : un service désactivé ne reçoit plus de lits ni d'admissions (R)
- [ ] Règles Services : historique conservé, pas de suppression physique (R)

- [ ] Règles Lits : code unique dans l'établissement (R)
- [ ] Règles Lits : attribution seulement si le lit est disponible à la date concernée (R)
- [ ] Règles Lits : un lit occupé ne peut pas devenir indisponible tant que le séjour n'est pas clôturé ou transféré (R)
- [ ] Règles Lits : un lit indisponible n'est jamais proposé pour une admission (R)
- [ ] Règles Lits : l'état du lit est cohérent avec les séjours actifs et les indisponibilités (R)
- [ ] Règles Séjours : admission possible seulement si patient et service existent et lit disponible (R)
- [ ] Règles Séjours : un patient n'a jamais deux séjours simultanés (R)
- [ ] Règles Séjours : sortie strictement postérieure à l'admission (R)
- [ ] Règles Séjours : un séjour terminé n'occupe plus de lit après sa sortie (R)
- [ ] Règles Séjours : les séjours terminés sont conservés pour les statistiques (R)
- [ ] Règles Séjours : modification ou clôture d'un séjour = mise à jour cohérente du lit (R)
- [ ] Règles Transferts : tout ou rien (R)
- [ ] Règles Transferts : lit de destination occupé ou indisponible refusé (R)
- [ ] Règles Transferts : conserver lit de départ, lit d'arrivée et heure (R)
- [ ] Règles Transferts : sortie impossible sur un séjour déjà terminé (R)

1.2 Modélisation

- [ ] Dessiner le MCD
- [ ] Dériver le MLD des 7 tables : Services, Lits, Patients, Sejours, Mouvements, Indisponibilites, Alertes
- [ ] Décider et écrire la **source de vérité de l'état d'un lit** : la colonne `etat` de Lits est maintenue par les opérations, Mouvements et Indisponibilités servent d'historique
- [ ] Prévoir une fonction de vérification de cohérence qui compare les deux

1.3 Champs et tables ajoutés (non écrits mais exigés par le cahier)

- [ ] `description` sur Services
- [ ] `description` sur Lits
- [ ] Colonne `regle` sur Alertes (la règle ayant déclenché l'alerte)
- [ ] Table `Parametres` (clé/valeur) : seuil d'indisponibilité (alerte 3), durée moyenne par défaut pour la simulation, etc.
- [ ] Documenter ces ajouts dans `docs/conception.md`

1.4 Simulation et interface

- [ ] Écrire sur papier les hypothèses de simulation (une admission fictive = nombre × durée moyenne de séjour en journées-lits)
- [ ] Maquetter l'interface : onglets Tableau de bord, Services, Lits, Patients, Séjours, Opérations, Indicateurs, Alertes, Simulation

---

Phase 2 : Base de données MySQL (1 jour)

2.1 Fichier `sql/schema.sql`

- [ ] Table `services` : `id_service` PK auto, `nom` unique obligatoire, `categorie` obligatoire, `description`, `seuil_occupation` (CHECK 1 à 100), `actif` (CHECK 0/1)
- [ ] Table `lits` : `id_lit` PK, `code_lit` unique obligatoire, `id_service` FK, `etat` (CHECK 3 valeurs), `motif_indisponibilite` facultatif, `description`
- [ ] Table `patients` : `id_patient` PK, `code_patient` unique obligatoire, `date_creation` DATETIME
- [ ] Table `sejours` : `id_sejour` PK, `id_patient` FK, `id_service` FK, `date_admission` obligatoire, `date_sortie` facultative, `statut` (CHECK)
- [ ] Table `mouvements` : `id_mouvement` PK, `id_sejour` FK, `id_lit` FK, `date_debut` obligatoire, `date_fin` facultative, `type_mouvement` (CHECK)
- [ ] Table `indisponibilites` : `id_indisponibilite` PK, `id_lit` FK, `date_debut` obligatoire, `date_fin` facultative, `motif` obligatoire
- [ ] Table `alertes` : `id_alerte` PK, `id_service` FK (si applicable), `type_alerte` obligatoire, `description` TEXT obligatoire, `date_alerte` obligatoire, `regle`
- [ ] Table `parametres` (clé/valeur)
- [ ] Toutes les clés étrangères en `ON DELETE RESTRICT`
- [ ] Index sur `sejours(id_patient, statut)`
- [ ] Index sur `mouvements(id_lit, date_debut, date_fin)`
- [ ] Index sur `indisponibilites(id_lit)`

2.2 Fichier `model/database.py`

- [ ] Connexion configurable depuis `config.py`
- [ ] Gestionnaire de transaction `with transaction():` (commit automatique, rollback sur exception)
- [ ] Fonctions d'exécution qui n'acceptent que des requêtes paramétrées (`%s`), aucune concaténation de chaînes
- [ ] Script de création de la base et des tables

2.3 Validation

- [ ] Test de démarrage : création de la base, des tables, reconnexion sans erreur

---

Phase 3 : Modèle, entités simples (2 jours)

> Pour chaque entité : CRUD, règles métier, puis tests, avant de passer à la suivante.

3.0 Exceptions métier

- [ ] Créer `model/exceptions.py` : `RegleMetierError`, `ConflitAffectationError`, `IncoherenceSejourError`, etc.

3.1 Module 1 : Services (`service_model.py`)

- [ ] Créer un service
- [ ] Consulter la liste des services
- [ ] Consulter le détail d'un service
- [ ] Modifier un service
- [ ] Désactiver un service (jamais de `DELETE`)
- [ ] Validation : nom obligatoire
- [ ] Validation : nom unique après normalisation
- [ ] Validation : catégorie obligatoire
- [ ] Validation : seuil entre 1 et 100
- [ ] Règle : un service inactif ne peut plus recevoir de lits
- [ ] Règle : un service inactif ne peut plus recevoir d'admissions
- [ ] **Test T1** : création d'un service valide, service enregistré
- [ ] **Test T2** : deux services avec le même nom, deuxième création refusée

3.2 Module 2 : Lits (`lit_model.py`)

- [ ] Ajouter un lit
- [ ] Modifier les informations d'un lit
- [ ] Rechercher un lit par son code
- [ ] Consulter les lits par service
- [ ] Consulter les lits par état
- [ ] Compter les lits disponibles, occupés et indisponibles (par service)
- [ ] Compter les lits disponibles, occupés et indisponibles (pour l'établissement)
- [ ] Refuser un lit dans un service inactif
- [ ] Refuser un code déjà existant
- [ ] **Test T3** : création d'un lit avec un code existant, opération refusée

3.3 Module 3 : Patients fictifs (`patient_model.py`)

- [ ] Génération automatique d'un code pseudonymisé (`P-0001`, ...), unique
- [ ] Date de création automatique
- [ ] Aucune donnée nominative (ni nom, ni adresse, ni diagnostic)
- [ ] Lister les patients

3.4 Indisponibilités (`indisponibilite_model.py`)

- [ ] Déclarer une indisponibilité (début, fin facultative, motif obligatoire)

- [ ] Refuser si le lit est occupé par un séjour en cours
- [ ] Refuser tout chevauchement avec une autre indisponibilité du même lit
- [ ] Clôturer une indisponibilité (remise en service)
- [ ] Mettre à jour `etat` et `motif_indisponibilite` du lit de façon cohérente
- [ ] Un lit indisponible n'est jamais proposé pour une admission

---

Phase 4 : Modèle, séjours et mouvements (3 jours)

> Cœur du projet. Toute opération multi-écritures passe par une transaction.

4.1 Admission (`sejour_model.py`)

- [ ] Vérifier : le patient existe
- [ ] Vérifier : le service existe et est actif
- [ ] Vérifier : le lit existe, appartient au service et est disponible
- [ ] Vérifier : le patient n'a pas déjà un séjour en cours
- [ ] Transaction : créer le séjour (`EN_COURS`)
- [ ] Transaction : créer le mouvement `ADMISSION`
- [ ] Transaction : passer le lit à `OCCUPE`
- [ ] Verrouiller le lit avec `SELECT ... FOR UPDATE`
- [ ] **Test T4** : admission dans un lit disponible, séjour créé, lit occupé
- [ ] **Test T5** : admission dans un lit indisponible, refusée
- [ ] **Test T6** : admission dans un lit déjà occupé, refusée

4.2 Sortie

- [ ] Refuser si le séjour n'est pas `EN_COURS` (sortie impossible sur séjour déjà terminé)
- [ ] Refuser si la date de sortie est antérieure ou égale à l'admission (alerte 4)
- [ ] Transaction : fermer le mouvement courant (`date_fin`)
- [ ] Transaction : créer le mouvement `SORTIE`
- [ ] Transaction : passer le séjour à `TERMINE` avec sa date de sortie
- [ ] Transaction : libérer le lit
- [ ] Si une indisponibilité est en cours sur le lit, le passer à `INDISPONIBLE` au lieu de `DISPONIBLE`
- [ ] **Test T7** : sortie avec une date antérieure à l'admission, refusée
- [ ] **Test T8** : sortie d'un séjour en cours, séjour terminé, lit libéré si utilisable

4.3 Transfert

- [ ] Vérifier : le séjour est en cours
- [ ] Vérifier : le lit de destination est disponible
- [ ] Vérifier : le lit de destination est différent du lit de départ
- [ ] Vérifier : le service de destination est actif

- [ ] Transaction : fermer le mouvement du lit de départ
- [ ] Transaction : libérer le lit de départ
- [ ] Transaction : créer le mouvement `TRANSFERT` sur le lit d'arrivée
- [ ] Transaction : occuper le lit d'arrivée
- [ ] Transaction : mettre à jour le service du séjour si besoin
- [ ] Rollback complet à la moindre erreur (jamais de patient sans affectation cohérente, jamais de double occupation)
- [ ] Conserver lit de départ, lit d'arrivée et heure du transfert
- [ ] **Test T9** : transfert vers un lit disponible, mouvement enregistré sans double occupation
- [ ] **Test T10** : transfert vers un lit occupé, refusé
- [ ] **Test T15** : échec d'un transfert (exception levée au milieu de la transaction), vérifier que rien n'a changé

4.4 Consultation et modification

- [ ] Historique des affectations d'un lit
- [ ] Historique des mouvements d'un séjour
- [ ] Liste des séjours avec filtres (statut, service, période)
- [ ] Modification d'un séjour (ex. correction de l'heure d'admission), cohérente avec le lit et les mouvements

4.5 Contrôle de cohérence global

- [ ] Détecter un lit `OCCUPE` sans séjour en cours
- [ ] Détecter un séjour en cours sans lit occupé
- [ ] Détecter deux séjours sur un même lit
- [ ] Détecter des mouvements qui se chevauchent
- [ ] Vérifier qu'elle retourne une liste vide sur des données saines

---

Phase 5 : Modèle, indicateurs (2 jours)

> Toutes les fonctions acceptent des filtres optionnels : `id_service`, `date_debut`, `date_fin`.

5.1 Calculs (`stats_model.py`)

- [ ] Nombre de lits par état (disponibles, occupés, indisponibles), par service et pour l'établissement
- [ ] Taux d'occupation instantané = occupés ÷ (occupés + disponibles) × 100
- [ ] Taux instantané : retourner « Non calculable » si aucun lit opérationnel
- [ ] Taux instantané : reconstruire l'état à un instant T passé via mouvements et indisponibilités (pas via la colonne `etat`)
- [ ] Taux sur une période = journées-lits occupées ÷ journées-lits opérationnelles × 100

- [ ] Période : calculer pour chaque lit l'intersection (en heures) entre chaque mouvement et la période
- [ ] Période : borner un séjour en cours par la fin de la période
- [ ] Période : journées opérationnelles = (nb lits × durée période − heures d'indisponibilité) ÷ 24
- [ ] Période : base sans séjour, résultat 0 % sans erreur
- [ ] Période : aucun lit opérationnel, « Non calculable »
- [ ] Durée moyenne des séjours terminés (ou « Non calculable »)

5.2 Autres indicateurs

- [ ] Nombre d'admissions sur une période
- [ ] Nombre de sorties sur une période
- [ ] Nombre de transferts
- [ ] Nombre de lits indisponibles
- [ ] Nombre de jours ou d'heures d'indisponibilité
- [ ] Service présentant le taux d'occupation le plus élevé
- [ ] Périodes durant lesquelles les seuils ont été dépassés
- [ ] Série temporelle : taux d'occupation jour par jour (pour le graphique d'évolution)

5.3 Tests

- [ ] **Test T11** : calcul d'occupation sur une base sans séjour, résultat défini sans erreur
- [ ] **Test T12** : aucun lit opérationnel dans un service, taux « Non calculable »
- [ ] Tests avec des cas dont le résultat est calculé à la main

---

Phase 6 : Modèle, alertes (1 jour)

6.1 Détection (`alerte_model.py`)

- [ ] **Alerte 1, occupation élevée** : taux instantané du service supérieur à son seuil
- [ ] **Alerte 2, saturation** : aucun lit disponible alors qu'il existe des lits opérationnels (cas « aucun lit opérationnel » traité à part)
- [ ] **Alerte 3, indisponibilité importante** : nombre de lits indisponibles supérieur au seuil configuré dans `Parametres`
- [ ] **Alerte 4, incohérence de séjour** : sortie refusée si date ⩽ admission, et alerte enregistrée pour garder la trace
- [ ] **Alerte 5, conflit d'affectation** : double affectation refusée, et alerte enregistrée

6.2 Contenu et gestion

- [ ] Chaque alerte contient : type, date et heure, service concerné, description, règle déclenchante
- [ ] Fonction `evaluer_alertes()` appelée après chaque admission

- [ ] Fonction `evaluer_alertes()` appelée après chaque sortie
- [ ] Fonction `evaluer_alertes()` appelée après chaque transfert
- [ ] Fonction `evaluer_alertes()` appelée après chaque changement d'indisponibilité
- [ ] Éviter les doublons (ne pas recréer une alerte active identique)
- [ ] Lister les alertes actives
- [ ] Consulter l'historique des alertes (avec filtre)
- [ ] **Test T13** : dépassement du seuil d'occupation, alerte générée

---

Phase 7 : Modèle, simulation (1 à 2 jours)

> Lectures et calculs en mémoire uniquement, aucune écriture en base.

7.1 Base (`simulation_model.py`)

- [ ] Calculer la situation initiale (capacité opérationnelle, taux d'occupation)

7.2 Scénarios

- [ ] **Scénario A, augmentation des admissions** : entrée = nombre d'admissions + période ; ajouter `n × durée moyenne de séjour` aux journées-lits occupées ; estimer le nouveau taux
- [ ] Scénario A : durée moyenne par défaut documentée si aucun séjour terminé n'existe
- [ ] **Scénario B, indisponibilité de lits** : entrée = service + nombre de lits + période ; retirer ces lits de la capacité ; recalculer le taux ; signaler le risque de saturation
- [ ] **Scénario C, réouverture de lits** : remettre en service k lits (k limité au nombre réellement indisponible) ; recalculer la capacité opérationnelle

7.3 Résultats attendus pour chaque scénario

- [ ] Situation initiale
- [ ] Paramètres du scénario
- [ ] Capacité opérationnelle après simulation
- [ ] Taux d'occupation estimé
- [ ] Alertes éventuelles
- [ ] Différence entre situation initiale et simulée

7.4 Documentation et test

- [ ] Rédiger dans la doc les hypothèses et limites (« estimation, pas une prédiction »)
- [ ] Expliquer comment l'effet des admissions simulées est représenté
- [ ] **Test T14** : simulation d'indisponibilité, capacité recalculée, tables identiques avant/après

---

Phase 8 : Jeu de données fictives (1 jour)

- [ ] Créer `data/seed.py`

- [ ] Fonction `reinitialiser_donnees()` : vider les tables dans l'ordre des dépendances puis réinsérer
- [ ] 4 services : médecine générale, chirurgie, pédiatrie, maternité
- [ ] 40 lits répartis entre les services
- [ ] 20 patients fictifs
- [ ] 30 séjours (terminés et en cours)
- [ ] Plusieurs transferts
- [ ] Plusieurs périodes d'indisponibilité
- [ ] Cas : un service à faible occupation
- [ ] Cas : un service proche de la saturation
- [ ] Cas : lits indisponibles
- [ ] Cas : séjours terminés et séjours en cours
- [ ] Cas : transferts valides
- [ ] Tentatives d'opérations invalides prévues dans un script de démonstration ou dans les tests (pas insérées en base)
- [ ] Les données passent par les fonctions du modèle (garantie de cohérence)
- [ ] Dates générées relativement à la date du jour
- [ ] Aucune donnée de vrais patients

Phase 9 : Tests automatisés (en continu, bilan sur 1 jour)

9.1 Mise en place

- [ ] Base de test séparée (`lits_test`)
- [ ] Réinitialisation avant chaque test (fixtures pytest)

9.2 Les 15 tests du cahier

- [ ] T1 : création d'un service valide
- [ ] T2 : deux services avec le même nom
- [ ] T3 : lit avec un code existant
- [ ] T4 : admission dans un lit disponible
- [ ] T5 : admission dans un lit indisponible
- [ ] T6 : admission dans un lit déjà occupé
- [ ] T7 : sortie avec date antérieure à l'admission
- [ ] T8 : sortie d'un séjour en cours
- [ ] T9 : transfert vers un lit disponible
- [ ] T10 : transfert vers un lit occupé
- [ ] T11 : calcul d'occupation sur une base sans séjour
- [ ] T12 : aucun lit opérationnel dans un service
- [ ] T13 : dépassement du seuil d'occupation

- [ ] T14 : simulation d'indisponibilité
- [ ] T15 : annulation ou échec d'un transfert

9.3 Tests complémentaires

- [ ] Service inactif (lit et admission refusés)
- [ ] Patient avec deux séjours simultanés
- [ ] Chevauchement d'indisponibilités
- [ ] Indisponibilité sur un lit occupé
- [ ] Cohérence globale après une série d'opérations
- [ ] Tableau récapitulatif (test / résultat attendu / résultat obtenu) pour le rapport

---

Phase 10 : Contrôleur (1 à 2 jours)

- [ ] Un contrôleur par module ou par groupe (référentiels, séjours, statistiques, simulation)
- [ ] Réception des actions de l'utilisateur
- [ ] Validation des entrées (champs vides, types, formats de date)
- [ ] Appel des méthodes du modèle
- [ ] Capture des exceptions métier et transformation en messages lisibles
- [ ] Actualisation de la vue
- [ ] Vérifier : aucune requête SQL ni calcul métier dans le contrôleur
- [ ] Vérifier : aucune requête SQL ni calcul métier dans la vue

---

Phase 11 : Vue PyQt (4 à 5 jours)

> Du plus simple au plus complexe. Teste chaque écran à la main avant le suivant.

- [ ] **11.1 Fenêtre principale** : `QTabWidget` ou menu latéral, barre de statut
- [ ] **11.2 Écran Services** : tableau
- [ ] Écran Services : formulaire créer/modifier
- [ ] Écran Services : bouton désactiver avec confirmation
- [ ] **11.3 Écran Lits** : tableau
- [ ] Écran Lits : filtres par service et par état
- [ ] Écran Lits : recherche par code
- [ ] Écran Lits : formulaire ajouter/modifier
- [ ] Écran Lits : compteurs disponibles / occupés / indisponibles
- [ ] Écran Lits : déclaration et fin d'indisponibilité
- [ ] Écran Lits : historique des affectations d'un lit sélectionné
- [ ] **11.4 Écran Patients** : liste et création d'un patient fictif
- [ ] **11.5 Écran Séjours / Opérations** : liste des séjours avec filtres et statut
- [ ] Formulaire Admission (patient, service, puis uniquement les lits disponibles de ce service)

- [ ] Formulaire Sortie (séjours en cours, date et heure)
- [ ] Formulaire Transfert (séjour en cours, lit de destination disponible)
- [ ] **11.6 Écran Indicateurs** : filtres service et période
- [ ] Écran Indicateurs : tous les indicateurs du module 5
- [ ] Écran Indicateurs : affichage de « Non calculable » quand c'est le cas
- [ ] **11.7 Écran Alertes** : alertes actives
- [ ] Écran Alertes : historique avec filtre
- [ ] **11.8 Tableau de bord** : nombre total de lits
- [ ] Tableau de bord : nombre de lits occupés
- [ ] Tableau de bord : nombre de lits disponibles
- [ ] Tableau de bord : nombre de lits indisponibles
- [ ] Tableau de bord : taux d'occupation par service
- [ ] Tableau de bord : admissions et sorties sur une période
- [ ] Tableau de bord : alertes actives
- [ ] Tableau de bord : durée moyenne des séjours terminés
- [ ] Graphique 1 (Matplotlib via `FigureCanvasQTAgg`) : comparaison du taux d'occupation entre services
- [ ] Graphique 2 (Matplotlib) : évolution du taux d'occupation sur une période
- [ ] Graphiques calculés à partir des données MySQL
- [ ] **11.9 Écran Simulation** : choix du scénario A, B ou C
- [ ] Écran Simulation : paramètres selon le scénario
- [ ] Écran Simulation : bouton « Simuler »
- [ ] Écran Simulation : tableau avant / après / différence
- [ ] Écran Simulation : alertes éventuelles
- [ ] Écran Simulation : texte rappelant les hypothèses et limites
- [ ] **11.10 Administration** : bouton « Réinitialiser les données de démonstration » avec confirmation
- [ ] **11.11 Messages** : confirmation et erreur partout (`QMessageBox`)

Phase 12 : Intégration et durcissement (2 jours)

12.1 Parcours complet à la main

- [ ] Démarrer l'application
- [ ] Réinitialiser les données de démonstration
- [ ] Faire une admission
- [ ] Faire un transfert
- [ ] Faire une sortie
- [ ] Déclarer une indisponibilité
- [ ] Constater une alerte

- [ ] Consulter le tableau de bord
- [ ] Lancer une simulation

12.2 Critères de réussite du cahier (section 11)

- [ ] L'application démarre correctement
- [ ] Les données sont enregistrées durablement dans MySQL (vérifier après redémarrage)
- [ ] Admission, sortie et transfert respectent les règles métier
- [ ] Aucun lit ne peut être affecté à plusieurs séjours simultanés
- [ ] Les calculs d'occupation sont cohérents
- [ ] Les alertes se déclenchent selon les règles définies
- [ ] Le tableau de bord présente des indicateurs calculés
- [ ] Les simulations n'altèrent pas les données réelles
- [ ] Tests et documentation fournis
- [ ] Je suis capable d'expliquer et de modifier mon code

12.3 Durcissement

- [ ] Lancer le contrôle de cohérence global (résultat vide attendu)
- [ ] Cas limite : base vide
- [ ] Cas limite : service sans lit
- [ ] Cas limite : dates identiques
- [ ] Cas limite : champs vides
- [ ] Tester sans connexion Internet

Phase 13 : Extensions facultatives (seulement si tout le reste est validé)

- [ ] **Extension 5** : seuils d'alerte configurables depuis l'interface (table `Parametres` déjà prête)
- [ ] **Extension 2** : historique détaillé des affectations et transferts pour chaque lit
- [ ] **Extension 1** : export des statistiques en CSV
- [ ] Extension 1 : export en PDF
- [ ] **Extension 3** : graphique d'évolution des admissions et sorties, ou comparaison des taux sur plusieurs périodes
- [ ] **Extension 4** : sauvegarde de plusieurs simulations et comparaison (table supplémentaire)

Phase 14 : Documentation et rendu (2 jours)

- [ ] Dossier de conception : règles métier, MCD/MLD, schéma SQL, diagramme MVC
- [ ] Conventions : formats de dates, conversion en journées-lits, hypothèses de simulation
- [ ] Manuel d'installation et d'utilisation (création de la base, lancement, réinitialisation)
- [ ] Rapport de tests (T1 à T15 et tests additionnels)

- [ ] Déclaration d'usage de l'IA finalisée (tâches, suggestions retenues, vérifications)
- [ ] Préparation à la soutenance : savoir expliquer et modifier en direct la transaction de transfert
- [ ] Préparation à la soutenance : savoir expliquer et modifier le calcul du taux sur période
- [ ] Préparation à la soutenance : savoir expliquer et modifier la détection de conflit d'affectation
- [ ] Relecture finale et dernier commit

Couverture du cahier des charges

Module / exigence	Phases
1. Services	3.1, 11.2
2. Lits	3.2, 3.4, 11.3
3. Patients et séjours	3.3, 4.1, 4.2, 4.4
4. Admission, sortie, transfert	4.1 à 4.3, 11.5
5. Indicateurs	5, 11.6
6. Alertes	6, 11.7
7. Tableau de bord	11.8
8. Simulation	7, 11.9
Schéma MySQL (7 tables)	2
Architecture MVC	3 à 11
Jeu de données	8
Tests T1 à T15	9
Règles d'utilisation de l'IA	0.3, 14
Extensions 1 à 5	13

Durée totale estimée : 3 à 4 semaines à rythme régulier. **Par où commencer :** Phase 0, puis Phase 1 (règles métier et schéma) avant d'écrire la moindre ligne de code.